FECAP – FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO

Curso de Ciência da Computação

6º Semestre – Turma 6º CCOMP

TENNIS ANALYTICS
TECNOLOGIAS E FERRAMENTAS DO PROTÓTIPO

Entrega 2 – Sistemas Embarcados e Robótica

Grupo D2LV

Daniel Moribe

Leonardo Santos

Lucas Gutierrez

Vitor Kanashiro

São Paulo

2026

1 O QUE É O PROJETO

O Tennis Analytics é um módulo eletrônico instalado na raquete de tênis que mede o movimento de cada golpe e mostra os resultados no celular. Ele faz parte do tema “Smart Tennis Arena” do Projeto Interdisciplinar, que propõe treinos de tênis com apoio de IoT e sensores.

É um projeto de Internet das Coisas (IoT): um objeto físico coleta dados e os envia a programas que os guardam e apresentam. Toda a visualização fica no aplicativo mobile; o grupo não fará interface web. A Figura 1 mostra o caminho do dado.

Figura 1 – Caminho do dado, da raquete ao servidor



Fonte: elaborado pelos autores (2026).

2 COMO FUNCIONA

Medir. O sensor MPU6050 mede quanto a raquete acelera e gira. O Arduino UNO R4 WiFi lê esses valores pelo protocolo I2C cerca de 100 vezes por segundo e executa o *firmware*, o programa gravado na placa.

Reconhecer. O *firmware* acompanha o gesto em quatro fases (máquina de estados): parado; preparação, quando a raquete começa a girar; impacto, quando a aceleração ultrapassa um limite; e recuperação, até o movimento terminar. No impacto, a aceleração em um dos eixos do sensor indica o tipo de golpe: *forehand*, com a palma da mão voltada para a frente, ou *backhand*, com as costas da mão voltadas para a frente. Ao final, o módulo calcula os picos de aceleração e de giro e estima a velocidade da raquete.

Enviar. Cada golpe vira uma mensagem curta de texto (JSON), enviada por Bluetooth de baixo consumo (BLE). A raquete não acessa a internet: o celular faz o papel de intermediário. O aplicativo mostra o golpe na hora e o guarda em uma fila, o que evita perdas caso a internet da quadra caia.

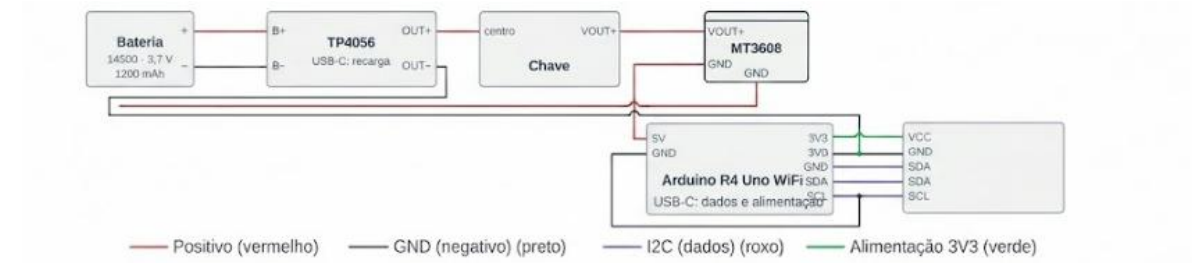
Guardar e mostrar. Da fila, os golpes seguem para o servidor, onde uma API (programa que atende pedidos de outros programas) confere os dados e os grava no banco de dados. Com esse histórico, o aplicativo monta painéis com gráficos e metas de desempenho.

3 MONTAGEM DO PROTÓTIPO

A eletrônica fica dentro do cabo da raquete, e não presa por fora do *grip*. O circuito reúne o Arduino UNO R4 WiFi, o sensor MPU6050, uma bateria e uma placa de carregamento (TP4056). A alimentação por bateria LiPo de 3,7 V substituiu a ideia de usar *power bank*.

Uma restrição do projeto: a bateria de 3,7 V, sozinha, não alimentou o UNO R4 WiFi nos testes. Para resolver, o grupo utilizou o conversor MT3608, que eleva a tensão da bateria até o valor exigido pela placa. A Figura 2 mostra o esquema de ligação.

Figura 2 – Esquema de ligação do protótipo



Fonte: elaborado pelos autores (2026).

4 TECNOLOGIAS E FERRAMENTAS

Quadro 1 – Tecnologias e ferramentas por parte do sistema

Parte	Tecnologia	O que é e para que serve
Raquete	Arduino UNO R4 WiFi	Microcontrolador que executa o *firmware*
Raquete	MPU6050 (GY-521)	Sensor de aceleração e de giro em três eixos
Raquete	Bateria LiPo de 3,7 V	Alimentação portátil
Raquete	TP4056 e MT3608	Carga da bateria por USB-C e conversor que eleva a tensão para a placa
Firmware	C/C++ e Arduino IDE	Linguagem do *firmware* e ferramenta que o grava
Comunicação	I2C, Bluetooth (BLE) e internet	Ligação do sensor, envio do golpe ao celular e do celular ao servidor
Aplicativo	TypeScript	Aplicativo mobile com histórico e gráficos
Servidor	API em Dart, Supabase e PostgreSQL	Recebe, confere e guarda os golpes
Apoio	Wokwi, VS Code, Git e GitHub	Simulação, edição do código e trabalho em equipe

Fonte: elaborado pelos autores (2026).

5 SITUAÇÃO ATUAL E LIMITAÇÕES

O caminho completo já funciona: o golpe é medido na raquete, passa pelo celular e chega ao banco de dados. O saque ficou fora desta versão, porque os dados coletados não permitiram diferenciá-lo do *forehand*. Dos 13 golpes gravados, os 10 *forehands* e *backhands* foram classificados corretamente. Esse resultado, porém, não mede a precisão real do sistema, pois o teste teve um único jogador e usou os mesmos dados da calibração.

O módulo também ainda não detecta se a bola foi atingida e não guarda os golpes feitos sem o celular conectado. Há ainda outras limitações conhecidas:

- O MPU6050 satura acima de ±16 g, e o impacto real da bola na raquete provavelmente ultrapassa esse valor. Se for preciso, o sensor H3LIS331DL, próprio para choques fortes, é a alternativa.
- O grupo substituiu o ESP32 pelo Arduino UNO R4 WiFi para simplificar a montagem do *hardware*.
- O LED e o buzzer ficaram fora do escopo atual e são ideias de melhoria futura.

APÊNDICE A – CÓDIGO-FONTE DO FIRMWARE DO PROTÓTIPO

```cpp
/*
  Smart Tennis Arena — firmware Uno R4 WiFi + MPU6050 (BLE)

  Implementa o plano-arduino-ble.md (tarefas A1 a A9):
    - Detecta o golpe a bordo (100 Hz) e envia só o resultado por BLE.
    - Classifica Forehand x Backhand e força (1 a 3).
    - Comandos do app: START, PAUSE, RESUME, STOP, PING, DBG:0, DBG:1.
    - Matriz de LEDs embutida como atuador local.

  Contrato BLE (igual ao protocolo.ts do app)
    Servico      4fafc201-1fb5-459e-8fcc-c5c9c331914b
    Dados  (notify) ...26a8   Arduino -> app
    Comando (write) ...26a9   app -> Arduino

    S:<F|B>,<picoG>,<picoGiro>,<forca>,<seq>   ex.: S:F,6.9,18.2,2,14
    E:<AGUARDANDO|COLETANDO|PAUSADO|PARADO>
    ERR:<texto>
    A:ax,ay,az|G:gx,gy,gz                  (so com DBG:1; ~40 bytes)

  A maquina de estados de deteccao e baseada na do Projeto1 (Swing Sense,
   passo2_maquina_estados_tacada_v5). Os LIMIARES abaixo sao PONTO DE PARTIDA:
  recalibre com a sua raquete e a sua montagem (plano, secao 4).

  Bibliotecas: Adafruit MPU6050, Adafruit Unified Sensor, ArduinoBLE,
  Arduino_LED_Matrix (ja vem no pacote "Arduino UNO R4 Boards").
*/

#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
```

```
#include <ArduinoBLE.h>
#include <Arduino_LED_Matrix.h>

// ====================== Configuracao ======================

#define SERVICE_UUID   "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHAR_DADOS_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a8"
#define CHAR_CMD_UUID   "beb5483e-36e1-4688-b7f5-ea07361b26a9"
#define NOME_BLE       "ArduinoR4-IMU"

// 1 = imprime CSV cru (t,ax,ay,az,gx,gy,gz) a 100 Hz no Serial, para a coleta
// de calibracao pelo cabo USB. Deixe 0 no uso normal.
#define SERIAL_RAW 0

// I2C: 100 kHz e o seguro para o cabo de ~50 cm ate o butt cap. Com o sensor
// no protoboard pode subir para 400000.
const uint32_t I2C_HZ = 100000;

const unsigned long PERIODO_AMOSTRA_US = 10000;   // 100 Hz
const unsigned long PERIODO_DEBUG_MS   = 100;     // stream A: a 10 Hz
const unsigned long INTERVALO_ENVIO_MS = 30;      // folga entre notificacoes BLE
const unsigned long TEMPO_LETRA_MS     = 500;     // quanto tempo F/B fica na
matriz

// ---------- Limiares de deteccao (Projeto1 v5 — RECALIBRAR) ----------
const float LIMIAR_GIRO_PREPARACAO = 1.5;    // rad/s
const float LIMIAR_ACEL_IMPACTO    = 20.0;   // m/s2
const float LIMIAR_ACEL_ESTAVEL    = 13.5;   // m/s2
const unsigned long TIMEOUT_PREPARACAO        = 1000;  // ms
const unsigned long TEMPO_ESTAVEL_RECUPERACAO = 150;   // ms
const unsigned long TIMEOUT_RECUPERACAO       = 2000;  // ms

// ---------- Classificacao (RECALIBRAR; depende da orientacao do sensor) ----------
```

```
const float LIMIAR_ACCX_BACKHAND = 14.0;      // m/s2, media de accX antes do impacto
const int   JANELA_AMOSTRAS      = 5;
const int   TAM_BUFFER           = 16;

// ---------- Forca por pico de giro (PLACEHOLDER — definir pelos tercis da coleta) ----------
const float FORCA_LIMIAR_1 = 12.0;          // rad/s: abaixo = fraco (1)
const float FORCA_LIMIAR_2 = 22.0;          // rad/s: abaixo = medio (2), acima = forte (3)

// ====================== Estado ======================

Adafruit_MPU6050 mpu;
ArduinoLEDMatrix matriz;

BLEService servico(SERVICE_UUID);
// 64 bytes de capacidade: S:/E:/ERR: tem no maximo 20; o A: de debug passa disso.
BLEStringCharacteristic dadosChar(CHAR_DADOS_UUID, BLERead | BLENotify, 64);
BLEStringCharacteristic cmdChar(CHAR_CMD_UUID, BLEWrite, 20);

// Estado do DISPOSITIVO (o que o app enxerga)
enum EstadoDisp { AGUARDANDO, COLETANDO, PAUSADO, PARADO };
EstadoDisp estadoDisp = AGUARDANDO;

// Estado da DETECCAO (interno). Nomes diferentes de PARADO de proposito.
enum EstadoDeteccao { OCIOSO, PREPARACAO, IMPACTO, RECUPERACAO };
EstadoDeteccao estadoDet = OCIOSO;

const char *nomeEstado(EstadoDisp e) {
  switch (e) {
    case AGUARDANDO: return "AGUARDANDO";
    case COLETANDO:  return "COLETANDO";
```

```
    case PAUSADO:    return "PAUSADO";
    default:         return "PARADO";
  }
}

unsigned long proximaAmostraUs = 0;
unsigned long ultimoDebugMs = 0;
unsigned long ultimoEnvioMs = 0;
unsigned long ultimoErroMpuMs = 0;
bool debugBle = false;

// Buffer circular de accX
float bufferAccX[TAM_BUFFER];
int posBuffer = 0;
int amostrasNoBuffer = 0;

// Golpe em andamento
unsigned long marcaTempo = 0;
unsigned long estavelDesde = 0;
char ladoAtual = 'F';
float accXJanela = 0;
float picoAcel = 0;
float picoGiro = 0;

unsigned int seq = 0;          // zera a cada boot; volta a 0 depois de 9999 (limite de 20
bytes)
unsigned long golpesPerdidos = 0;  // golpes detectados sem central conectada

// Fila de saida BLE (uma notificacao por vez, com folga entre elas)
const int TAM_FILA = 8;
char fila[TAM_FILA][24];
int filaIni = 0;
int filaN = 0;
```

```
// Comando recebido (o handler so guarda; quem executa e o loop)
char cmdPendente[24];
volatile bool cmdNovo = false;

// Matriz
unsigned long letraAteMs = 0;
bool mostrandoLetra = false;

// ====================== Matriz de LEDs ====================

const char *ICONE_AGUARDANDO[8] = {
  ".....", ".....", ".....", "....####....",
  "....####....", ".....", ".....", "....."
};
const char *ICONE_COLETANDO[8] = {
  ".....", "..########..", "..########..", "..########..",
  "..########..", "..########..", "..########..", "....."
};
const char *ICONE_PAUSADO[8] = {
  ".....", "...###..###.", "...###..###.", "...###..###.",
  "...###..###.", "...###..###.", "...###..###.", "....."
};
const char *ICONE_PARADO[8] = {
  ".....", "..########..", "..#......#..", "..#......#..",
  "..#......#..", "..#......#..", "..########..", "....."
};
const char *LETRA_F[8] = {
  "...#####....", "...#........", "...#........", "...####.....",
  "...#........", "...#........", "...#........", "...#........"
};
const char *LETRA_B[8] = {
  "...####.....", "...#...#....", "...#...#....", "...####.....",
  "...#...#....", "...#...#....", "...#...#....", "...####....."
};
```

```
void desenhar(const char *linhas[8]) {
  uint32_t quadro[3] = {0, 0, 0};
  for (int r = 0; r < 8; r++) {
    for (int c = 0; c < 12; c++) {
      if (linhas[r][c] == '#') {
        int i = r * 12 + c;
        quadro[i / 32] |= (1UL << (31 - (i % 32)));
      }
    }
  }
  matriz.loadFrame(quadro);
}

void desenharIconeEstado() {
  switch (estadoDisp) {
    case AGUARDANDO: desenhar(ICONE_AGUARDANDO); break;
    case COLETANDO:  desenhar(ICONE_COLETANDO);  break;
    case PAUSADO:    desenhar(ICONE_PAUSADO);    break;
    default:         desenhar(ICONE_PARADO);     break;
  }
}

void mostrarLetra(char lado) {
  desenhar(lado == 'F' ? LETRA_F : LETRA_B);
  mostrandoLetra = true;
  letraAteMs = millis() + TEMPO_LETRA_MS;
}

void atualizarMatriz() {
  if (mostrandoLetra && (long)(millis() - letraAteMs) >= 0) {
    mostrandoLetra = false;
    desenharIconeEstado();
  }
```

```
}

// ====================== Fila BLE ======================

void enfileirar(const char *msg) {
  if (filaN == TAM_FILA) {           // cheia: descarta a mais antiga
    filaIni = (filaIni + 1) % TAM_FILA;
    filaN--;
  }
  int pos = (filaIni + filaN) % TAM_FILA;
  strncpy(fila[pos], msg, sizeof(fila[pos]) - 1);
  fila[pos][sizeof(fila[pos]) - 1] = '\0';
  filaN++;
}

void enviarFila() {
  if (filaN == 0 || !BLE.connected()) return;
  if (millis() - ultimoEnvioMs < INTERVALO_ENVIO_MS) return;
  dadosChar.writeValue(fila[filaIni]);
  filaIni = (filaIni + 1) % TAM_FILA;
  filaN--;
  ultimoEnvioMs = millis();
}

void limparFila() {
  filaIni = 0;
  filaN = 0;
}

// ====================== Deteccao ======================

void reiniciarDeteccao() {
  estadoDet = OCIOSO;
  picoAcel = 0;
```

```
  picoGiro = 0;
  estavelDesde = 0;
}

void mudarEstado(EstadoDisp novo) {
  if (novo == estadoDisp) return;
  estadoDisp = novo;
  reiniciarDeteccao();
  char msg[24];
  snprintf(msg, sizeof(msg), "E:%s", nomeEstado(novo));
  enfileirar(msg);
  if (!mostrandoLetra) desenharIconeEstado();
  Serial.print("Estado: ");
  Serial.println(nomeEstado(novo));
}

void guardarNoBuffer(float v) {
  bufferAccX[posBuffer] = v;
  posBuffer = (posBuffer + 1) % TAM_BUFFER;
  if (amostrasNoBuffer < TAM_BUFFER) amostrasNoBuffer++;
}

float mediaUltimas(int n) {
  if (n > amostrasNoBuffer) n = amostrasNoBuffer;
  if (n == 0) return 0;
  float soma = 0;
  for (int i = 1; i <= n; i++) {
    soma += bufferAccX[(posBuffer - i + TAM_BUFFER) % TAM_BUFFER];
  }
  return soma / n;
}

int classificarForca(float giro) {
  if (giro < FORCA_LIMIAR_1) return 1;
```

```
  if (giro < FORCA_LIMIAR_2) return 2;
  return 3;
}

void reportarGolpe() {
  float picoG = picoAcel / 9.81f;
  if (picoG > 99.9f) picoG = 99.9f;
  float giro = picoGiro > 99.9f ? 99.9f : picoGiro;
  int forca = classificarForca(picoGiro);

  seq = (seq + 1) % 10000;

  // Feedback local primeiro (nao depende de BLE)
  mostrarLetra(ladoAtual);

  char msg[24];
   snprintf(msg, sizeof(msg), "S:%c,%.1f,%.1f,%d,%u", ladoAtual, picoG, giro, forca,
seq);
  if (BLE.connected()) {
    enfileirar(msg);
  } else {
    golpesPerdidos++;
  }

  Serial.print("GOLPE,");
  Serial.print(ladoAtual);
  Serial.print(",accX_janela=");
  Serial.print(accXJanela, 2);
  Serial.print(",pico_acel=");
  Serial.print(picoAcel, 1);
  Serial.print(",pico_giro=");
  Serial.print(picoGiro, 1);
  Serial.print(",forca=");
  Serial.print(forca);
```

```
  Serial.print(",seq=");
  Serial.print(seq);
  Serial.print(" -> ");
  Serial.println(msg);
}

void processarDeteccao(const sensors_event_t &a, const sensors_event_t &g,
unsigned long agora) {
  float accelMag = sqrtf(a.acceleration.x * a.acceleration.x +
                         a.acceleration.y * a.acceleration.y +
                         a.acceleration.z * a.acceleration.z);
  float giroMag = sqrtf(g.gyro.x * g.gyro.x +
                        g.gyro.y * g.gyro.y +
                        g.gyro.z * g.gyro.z);

  switch (estadoDet) {

    case OCIOSO:
      if (giroMag > LIMIAR_GIRO_PREPARACAO) {
        estadoDet = PREPARACAO;
        marcaTempo = agora;
        picoAcel = 0;
        picoGiro = 0;
      }
      break;

    case PREPARACAO:
      if (giroMag > picoGiro) picoGiro = giroMag;
      if (accelMag > LIMIAR_ACEL_IMPACTO) {
        if (accelMag > picoAcel) picoAcel = accelMag;
        accXJanela = mediaUltimas(JANELA_AMOSTRAS);
        ladoAtual = (accXJanela > LIMIAR_ACCX_BACKHAND) ? 'B' : 'F';
        estadoDet = IMPACTO;
        marcaTempo = agora;
```

```
    } else if (agora - marcaTempo > TIMEOUT_PREPARACAO) {
      estadoDet = OCIOSO;
    }
    break;

  case IMPACTO:
    if (accelMag > picoAcel) picoAcel = accelMag;
    if (giroMag > picoGiro) picoGiro = giroMag;
    estadoDet = RECUPERACAO;
    marcaTempo = agora;
    estavelDesde = 0;
    break;

  case RECUPERACAO:
    // o pico real vem ~100 ms depois do cruzamento: continua rastreando
    if (accelMag > picoAcel) picoAcel = accelMag;
    if (giroMag > picoGiro) picoGiro = giroMag;

    if (accelMag < LIMIAR_ACEL_ESTAVEL) {
      if (estavelDesde == 0) {
        estavelDesde = agora;
      } else if (agora - estavelDesde > TEMPO_ESTAVEL_RECUPERACAO) {
        reportarGolpe();
        estadoDet = OCIOSO;
      }
    } else {
      estavelDesde = 0;
    }

          if (estadoDet == RECUPERACAO && agora - marcaTempo >
TIMEOUT_RECUPERACAO) {
      reportarGolpe();
      estadoDet = OCIOSO;
    }
```

```
      break;
    }
  }

// ====================== Amostragem ====================

void amostrar() {
  sensors_event_t a, g, temp;
  if (!mpu.getEvent(&a, &g, &temp)) {
    if (millis() - ultimoErroMpuMs > 2000) {
      ultimoErroMpuMs = millis();
      enfileirar("ERR:MPU");
      Serial.println("ERRO: leitura do MPU falhou");
    }
    return;
  }

  guardarNoBuffer(a.acceleration.x);

#if SERIAL_RAW
  Serial.print(millis()); Serial.print(',');
  Serial.print(a.acceleration.x, 3); Serial.print(',');
  Serial.print(a.acceleration.y, 3); Serial.print(',');
  Serial.print(a.acceleration.z, 3); Serial.print(',');
  Serial.print(g.gyro.x, 3); Serial.print(',');
  Serial.print(g.gyro.y, 3); Serial.print(',');
  Serial.println(g.gyro.z, 3);
#endif

  unsigned long agora = millis();

  if (estadoDisp == COLETANDO) {
    processarDeteccao(a, g, agora);
  }
```

```
  // Stream de debug (so se o app pediu DBG:1)
    if (debugBle && BLE.connected() && agora - ultimoDebugMs >=
PERIODO_DEBUG_MS) {
    ultimoDebugMs = agora;
    char msg[48];
    snprintf(msg, sizeof(msg), "A:%.1f,%.1f,%.1f|G:%.1f,%.1f,%.1f",
            a.acceleration.x, a.acceleration.y, a.acceleration.z,
            g.gyro.x, g.gyro.y, g.gyro.z);
    // so entra na fila se houver folga, para nao atrasar S:/E:
    if (filaN < TAM_FILA - 2) enfileirar(msg);
  }
}

// ===================== Comandos ====================

void aoEscreverComando(BLEDevice central, BLECharacteristic caracteristica) {
  String v = cmdChar.value();
  v.trim();
  v.toUpperCase();
  v.toCharArray(cmdPendente, sizeof(cmdPendente));
  cmdNovo = true;
}

void executarComando(const char *cmd) {
  Serial.print("Comando: ");
  Serial.println(cmd);

  if (strcmp(cmd, "START") == 0) {
    // AGUARDANDO/PARADO/PAUSADO -> COLETANDO
    if (estadoDisp == COLETANDO) {
      reiniciarDeteccao();
      enfileirar("E:COLETANDO");
    } else {
```

```
      mudarEstado(COLETANDO);
    }
  } else if (strcmp(cmd, "PAUSE") == 0) {
    if (estadoDisp == COLETANDO) mudarEstado(PAUSADO);
    else enfileirar("ERR:PAUSE");
  } else if (strcmp(cmd, "RESUME") == 0) {
    if (estadoDisp == PAUSADO) mudarEstado(COLETANDO);
    else enfileirar("ERR:RESUME");
  } else if (strcmp(cmd, "STOP") == 0) {
    if (estadoDisp == PARADO) {
      enfileirar("E:PARADO");
    } else {
      mudarEstado(PARADO);
    }
  } else if (strcmp(cmd, "PING") == 0) {
    char msg[24];
    snprintf(msg, sizeof(msg), "E:%s", nomeEstado(estadoDisp));
    enfileirar(msg);
  } else if (strcmp(cmd, "DBG:1") == 0) {
    debugBle = true;
  } else if (strcmp(cmd, "DBG:0") == 0) {
    debugBle = false;
  } else {
    enfileirar("ERR:CMD");
  }
}

// ====================== Conexao BLE ====================

void aoConectar(BLEDevice central) {
  Serial.print("Conectado ao central: ");
  Serial.println(central.address());
  limparFila();
  debugBle = false;
```

```
  // Nova conexao: espera o START do app. Sem E: aqui — o app ainda pode nao
  // ter assinado as notificacoes; ele sincroniza com PING.
  estadoDisp = AGUARDANDO;
  reiniciarDeteccao();
  if (!mostrandoLetra) desenharIconeEstado();
}

void aoDesconectar(BLEDevice central) {
  Serial.println("Central desconectado: parada segura. Anunciando de novo...");
  limparFila();
  debugBle = false;
  cmdNovo = false;
  estadoDisp = PARADO;
  reiniciarDeteccao();
  if (!mostrandoLetra) desenharIconeEstado();
  BLE.advertise();
}

// ===================== setup / loop =====================

void setup() {
  Serial.begin(115200);
  while (!Serial && millis() < 3000);

  matriz.begin();
  desenharIconeEstado();

  Wire.begin();
  Wire.setClock(I2C_HZ);

  if (!mpu.begin(0x68, &Wire)) {
    Serial.println("ERRO: MPU-6050 nao encontrado! Confira SDA/SCL/VCC/GND.");
    while (1) delay(10);
  }
```

```
  mpu.setAccelerometerRange(MPU6050_RANGE_16_G);
  mpu.setGyroRange(MPU6050_RANGE_2000_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);   // referencia dos limiares

  if (!BLE.begin()) {
    Serial.println("ERRO: falha ao iniciar BLE!");
    while (1) delay(10);
  }

  BLE.setLocalName(NOME_BLE);
  BLE.setDeviceName(NOME_BLE);
  BLE.setAdvertisedService(servico);

  servico.addCharacteristic(dadosChar);
  servico.addCharacteristic(cmdChar);
  BLE.addService(servico);

  BLE.setEventHandler(BLEConnected, aoConectar);
  BLE.setEventHandler(BLEDisconnected, aoDesconectar);
  cmdChar.setEventHandler(BLEWritten, aoEscreverComando);

  dadosChar.writeValue("E:AGUARDANDO");
  BLE.advertise();

  Serial.println("Pronto. BLE anunciando como " NOME_BLE);
  proximaAmostraUs = micros();
}

void loop() {
  BLE.poll();

  if (cmdNovo) {
    cmdNovo = false;
    char copia[24];
```

```
    strncpy(copia, cmdPendente, sizeof(copia));
    copia[sizeof(copia) - 1] = '\0';
    executarComando(copia);
  }

  // 100 Hz, independente do BLE. Se atrasar muito, ressincroniza em vez de
  // disparar leituras em rajada.
  unsigned long agoraUs = micros();
  if ((long)(agoraUs - proximaAmostraUs) >= 0) {
    if ((long)(agoraUs - proximaAmostraUs) > 50000) proximaAmostraUs = agoraUs;
    proximaAmostraUs += PERIODO_AMOSTRA_US;
    amostrar();
  }

  enviarFila();
  atualizarMatriz();
}
```